

Föreläsning 22

Ändrad
15 Oct
17:49

Tutti frutti

Nu blir det blandad frukt, litet av varje från det som återstår.

while (n--)

Ett sätt att repetera en loop n gånger är `while (n--)`.
Exempel, skriv ut n blanktecken:

```
n = (int)floor(100 * sin(x));
while (n--)
    putchar(' ');
    putchar('*');
```

Metoden fungerar eftersom falskt = 0 och sant är allt som inte är noll.

■ Även strängar slutar på noll. Mycket ofta ser man därför följande, eftersom det är så frestande kortfattat.
Programsnutten skriver ut en sträng.

```
char s[] = "Fem meter upp i luften", *p = s;
while (*p)
    putchar(*p++);
    putchar('\n');
```

Man kan utnyttja samma finesse med vanlig indexering:

```
char s[] = "Fem meter upp i luften", i = 0;
while (s[i])
    putchar(s[i++]);
    putchar('\n');
```

break (och return)

För välja vilken kod som ska exekveras, använder vi loopar och val-satser. De loopar som vi gått igenom är tillräckliga för varje tänkbar situation -- detta kan bevisas!

Däremot kan koden ibland bli snyggare om vi "sätter oss över" looparnas lag och ordning. Vi kan avbryta en loop i förtid med kommandot `break`, och vi kan avbryta ett loopvarv genom kommandot `continue`.

Det kan tänkas användas när oförutsedda saker kan inträffa *mitt inne i* en loop.

```
while(p) {
    <SATSER 1>
    if(nåt_oförutsett_har_inträffat)
        break;
    <SATSER 2>
}
```

Ovanstående kan å andra sidan alltid skrivas om till

```
while(p && !nåt_oförutsett_har_inträffat) {  
    <SATSER 1>  
    if(!nåt_oförutsett_har_inträffat)  
        <SATSER 2>  
}
```

Kommandot `return` behöver inte stå sist i en funktion. Det kan stå mitt inne i en loop till exempel. Vilket i princip ger samma effekt som en `break`-sats.

Det har jag utnyttjat i tidigare programexempel. Exempelvis i funktionen `equal1()` i [föreläsning 18](#).

En variant som inte bryter loopen på onaturligt sätt finns också, i funktionen `equal4()`.

`continue`

Kommandot `continue` avbryter ett loopvarv. Jag har inget minne av att jag någonsin använt mig av den möjligheten. Återigen handlar det i så fall om en situation där man önskar avbryta mitt inne i loopen.

```
while(p) {  
    <SATSER 1>  
    if(nåt_oförutsett_har_inträffat)  
        continue;  
    <SATSER 2>  
}
```

Återigen bör man hellre göra en omskrivning, denna gång blir det inte ens mer att skriva:

```
while(p) {  
    <SATSER 1>  
    if(!nåt_oförutsett_har_inträffat)  
        <SATSER 2>  
}
```

■ Kommandona `break` och `continue` skall användas sparsamt! Koden är lättare att läsa och förstå om looparna fungerar normalt.

`goto`

I nödfall kan man använda `break` och `continue` för att bryta den vanliga ordningen. Däremot behöver du (troligen) aldrig `goto`. Med `goto` kan du hoppa till vilken rad som helst i funktionen.

Jag tänker inte ens ge ett exempel. Jag kommer inte på nåt! Hade inte `break` och `continue` funnits, skulle `goto` kunna utnyttjas till ungefär samma sak.

Alltså: Använd inte `goto`! Läs gärna vad [Bilting/Skansholm](#) säger om `goto`, `break` och `continue` i kapitel 5.9.

`exit()`

Funktionen `exit()` avbryter hela programkörningen. Det blir som om huvudprogrammet tar slut.

Det är bara för stora katastrofer som ett riktigt program ska tillgripa `exit`. Att anropa `exit` är att erkänna att man gett upp.

I [gruppövning 3](#) visar jag hur man kan använda `exit` för att implementera `assert` -- en rudimentär felkontroll som bör ersättas med något användarvänligare i ett färdigt program.

Bitoperationer

Det finns bitoperationer i C. Logiska operationer på bit-nivå:

```
char i = 12; /* = 00001100 */
char j = 10; /* = 00001010 */
char k;

k = ~j;      /* k = 11110101 = 245 */
k = j & i;    /* k = 00001000 = 8 */
k = j | i;    /* k = 00001110 = 14 */
k = j ^ i;    /* k = 00000110 = 6 (XOR) */

k = j << 3;   /* k = 01010000 (vänsterskift) */
k = j >> 2;   /* k = 00000010 (högerskift) */
```

Högerskiften *kan* skifta in en etta om talet är negativt. Detta är implementationsberoende.

Miniräknare med flexibel inmatning

Man kan använda `scanf()` för att läsa in ett flyttal. Men saker kan gå fel, och programmet kan krascha. Till exempel om användaren matar in tecken i stället för siffror.

Det finns ett ganska enkelt sätt i C att göra inmatning säkrare: läs in hela raden med `gets()`. Undersök därefter vad som stod på raden genom att anropa till exempel `sscanf()` och `strcmp()` flera gånger, med olika format.

På detta sätt kan du dessutom få en inmatning som klarar av att känna igen helt olika sorters rader.

Jag har skrivit ett demonstrationsprogram som är en avancerad variant av er laboration 2. Lägg märke till att indata inte är begränsat till att alltid se ut på exakt samma sätt. Och att programmet kan ge intressanta felkommentarer.

```
1  = 0
2  > 5+5
3  = 10
4  > 2.2 - 5
5  = -2.8
6  > *6
7  = -16.8
8  > sqr
9  = 282.24
10 > sin 3.14
11 = 0.00159265
12 > log 12
13 Unknown funktion: log
14 = 12
15 > cube
16 = 1728
17 > time
```

```

18 Time is 11:20
19 = 1728
20 > 3 ^ 3
21 = 27
22 > quit
23 bye

```

Källkod:

```

1  #include <stdio.h>
2  #include <time.h>
3  #include <math.h>
4
5  extern int printf();
6  extern int fprintf();
7  extern int scanf();
8  extern int sscanf();
9  extern char *gets();
10 extern time_t *time();
11
12 void printtime()
13 {
14     time_t t;
15     struct tm *now;
16     time(&t);
17     now = localtime(&t);
18     printf("Time is ");
19     printf("%02d:%02d\n", now->tm_hour, now->tm_min);
20 }
21
22
23 /* Ex: evaluate_function("sin", 1.3) == sin(1.3) */
24 double evaluate_function(char function[], double x)
25 {
26     if (strcmp(function, "sin") == 0)
27         x = sin(x);
28     else if (strcmp(function, "cos") == 0)
29         x = cos(x);
30     else if (strcmp(function, "tan") == 0)
31         x = tan(x);
32     else if (strcmp(function, "ln") == 0)
33         x = log(x);
34     else if (strcmp(function, "sqrt") == 0)
35         x = sqrt(x);
36     else if (strcmp(function, "sqr") == 0)
37         x = x * x;
38     else if (strcmp(function, "cube") == 0)
39         x = x * evaluate_function("sqr", x);
40     else
41         fprintf(stderr, "Unknown funktion: %s\n", function);
42     return x;
43 }
44
45
46 /* Ex: evaluate_expression(12, '+', 1.3) == 13.3 */
47 double evaluate_expression(double y, char operator, double z)
48 {
49     double x = 0;
50
51     switch(operator) {
52     case '+': x = y + z;
53             break;
54     case '-': x = y - z;
55             break;
56     case '*': x = y * z;
57             break;
58     case '/': x = y / z;
59             break;
60     case '^': x = exp(z * log(y));
61             break;
62     default: fprintf(stderr, "Unknown operator: '%c'\n", operator);
63             break;
64     }
65     return x;
66 }
67
68
69 double evaluate(char line[], double x)
70 {
71     double y = 0, z = 0;
72     char op[2], function[5];
73

```

```

74     if (sscanf(line, "%lg %ls %lg", &y, op, &z) == 3)
75         x = evaluate_expression(y, op[0], z);
76     else if (sscanf(line, "%ls %lg", op, &z) == 2)
77         x = evaluate_expression(x, op[0], z);
78     else if (sscanf(line, "%4s %lg", function, &y) == 2)
79         x = evaluate_function(function, y);
80     else if (sscanf(line, "%4s", function) == 1)
81         x = evaluate_function(function, x);
82     else if (sscanf(line, "%lg", &y) == 1)
83         x = y;
84     else
85         fprintf(stderr, "Fattar inte ...!\n");
86     return x;
87 }
88
89 void calculator()
90 {
91     double x = 0;
92     char line[200], done = 0;
93
94     do {
95         printf("= %g\n", x);
96         printf("> ");
97         if (gets(line) == NULL)
98             done = 1;
99         else if (strcmp(line, "quit") == 0)
100             done = 1;
101         else if (strcmp(line, "time") == 0)
102             printtime();
103         else
104             x = evaluate(line, x);
105     } while (!done);
106     printf("bye\n");
107 }
108
109 void main()
110 {
111     calculator();
112 }
113

```

I källkoden definieras sex funktioner:

■ `printtime()` skriver ut aktuellt klockslag. Funktionerna som anropas, och typerna som används, finns i biblioteket `time.h`. Jag har stulit ett exempel från Bilting/Skansholm, kapitel 10.9 (min manual var otroligt svårtolkad!)

Det enda som är principiellt nytt för er är att där förekommer en typ som heter `struct tm` -- det är ett alternativt sätt att namnge `struct`ar. Programmeraren har inte skrivit exakt som vi i sin typdefinition.

Istället för

```

typedef struct {
    ...
    int  tm_min;
    int  tm_hour;
    ...
} tm;

```

vilket skulle definierat typnamnet `tm`, har programmeraren skrivit:

```

struct tm {
    ...
    int  tm_min;
    int  tm_hour;
    ...
};

```

vilket definierar typnamnet `struct tm`. Förutom i namnet finns inga skillnader mellan typerna.

Jag -- och två experter jag talat med -- rekommenderar `typedef`-varianten. `struct`-varianten är en äldre tradition som lever kvar.

■ `evaluate_function()` kan anropas på följande sätt:

```
A = evaluate_function("sin", 1.2);
```

och returnerar då `sin(1.2)`.

Funktionen skriver ett felmeddelande om den anropas med ett funktionsnamn den inte känner igen. Felmeddelandet skrivs ut på filen `stderr` -- detta är normalt en synonym för `stdout` -- texten kommer på bildskärmen som vanligt. Men det finns en konvention om att felmeddelanden ska placeras på `stderr`. Detta gäller även andra budskap som ligger utanför det normala utdata -- inte bara felmeddelanden.

Funktionsnamnet testas i en `if`-stege med hjälp av `strcmp()`. Variabeln `x` blir senare returvärdet.

Funktionen anropar sig själv på rad 39. Detta är tillåtet. Det kallas "rekursion": anropet är "rekursivt" och funktionen är "rekursiv". (det hade förstås varit enklare att skriva `x = x * x * x`, men jag ville demonstrera rekursion).

Rekursion fungerar på följande sätt: låtsas att det rekursiva anropet egentligen anropar en annan funktion som heter `evaluate_function2()`, men som ser exakt likadan ut som `evaluate_function()`.

■ `evaluate_expression()` kan anropas på följande sätt:

```
A = evaluate_expression(12, '+', 1.2);
```

och returnerar då `13.2`.

■ `evaluate()` provar olika tolkningar av inmatningsraden. Formatsträngarna `"%4s"` och `"%1s"` betyder läs in högst fyra respektive ett tecken till strängen.

■ `calculator()` snurrar runt och läser in och gör beräkningar tills användaren skriver `quit` (rad 98) eller `[Control]+[d]` (rad 100).

■ `main()` anropar `calculator()`

Mer om rekursion -- fakultet

Det klassiska exemplet på rekursion är *fakultet*. Fakulteten av n är produkten av alla positiva heltal tal upp till och med n . Dessutom räknas fakulteten av noll som ett (det är bara en konvention, det finns ingen logik i det).

På era miniräknare finns en knapp med ett utropstecken, den beräknar fakulteten. Man brukar skriva $n!$ för fakulteten

av n .

Fakulteten kan definieras rekursivt: fakulteten av n är lika med fakulteten av $n - 1$ -- gånger n . Exempelvis är alltså $5! = 4! * 5$.

Den rekursiva definitionen skjuter problemet framför sig -- för att beräkna fakulteten, måste vi beräkna en annan fakultet. De rekursiva anropen tar dock slut vid noll (noll-fakultet är ett).

Här följer två definitioner av fakultet. Den första *iterativ* (så heter det när man använder loopar) och den andra *rekursiv*.

```
6 unsigned long fac1(unsigned long n)
7 {
8     unsigned long i, product = 1;
9
10    for (i = 2; i <= n; i++)
11        product *= i;
12    return product;
13 }
14
15 unsigned long fac2(unsigned long n)
16 {
17     if (n == 0)
18         return 1;
19     else
20         return n * fac2(n - 1);
21 }
```

Nästa lektion

